

Νευρωνικά Δίκτυα, 1η Εργασία

Ιωάννης Κέλπης AEM:10300

Νοέμβριος 2023

Περίληψη

Σκοπός της εργασίας είναι να γραφεί και να μελετηθεί ένα νευρωνικό δίκτυο πολυστρωματικού perceptron, το οποίο θα εκπαιδεύεται με τον αλγόριθμο back propagation. Η γλώσσα που επιλέχθηκε για το συγκεκριμένο project είναι η Python (version 3.11.5).

Η βάση δεδομένων που επιλέχθηκε είναι η Cifar10, χρησιμοποιήθηκε ολόκληρη.

Η υλοποίηση του Νευρωνικού Δικτύου, έγινε με την βοήθεια της βιβλιοθήκης Pytorch, η οποία είναι μια open source βιβλιοθήκη της python, που προσφέρει, εκτός των άλλων, το module "torch.nn" για την κατασκευή και την εκπαίδευση Νευρωνικών Δικτύων. Το module αυτό περιλαμβάνει προκαθορισμένα επίπεδα (layers), συναρτήσεις ενεργοποίησης (activation functions), και αλγορίθμους βελτιστοποίησης, καθιστώντας την δημιουργία ενός νευρωνικού δικτύου, αρκετά πιο εύκολη!

Αφού δημιουργήθηκε το νευρωνικό δίκτυο, δοκιμάστηκαν και μελετήθηκαν διάφοροι τρόποι εκπαίδευσης, και διάφορες τιμές τόσο για τον αριθμό των νευρώνων στο κρυφό επίπεδο, όσο και για την τιμή του learning rate, παράμετροι που επηρεάζουν σημαντικά την ακρίβεια του νευρωνικού δικτύου.

Chapter 1

Παρουσίαση και ανάλυση του κώδικα

Σε αυτό το κεφάλαιο θα μελετήσουμε και θα αναλύσουμε τον κώδικα που είναι υπεύθυνος για την δημιουργία και την εκπαίδευση του Νευρωνικού μας δικτύου.

Πρώτο και σημαντικότερο βήμα, είναι η εισαγωγή των απαραίτητων βιβλιοθηκών.

- "torch": βασική βιβλιοθήκη του PyTorch.
- "torch.nn": παρέχει την βάση για την κατασκευή και την εκπαίδευση του νευρωνικού δικτύου μας.
- "torch.optim": παρέχει optimizers για την εκπαίδευση του μοντέλου. (στην περίπτωση μας τον SGD)
- "torchvision": παρέχει εργαλεία και datasets. (στην περίπτωση μας το dataset Cifar10)
- "torch.utils.data": παρέχει εργαλεία για την διαχείριση των δεδομένων, και την δημιουργία data loaders.
- "matplotlib.pyplot": παρέχει εργαλεία για την γραφική αναπαράσταση των αποτελεσμάτων του δικτύου μας.

```
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision
import torchvision.datasets as datasets
import torchvision.transforms as transforms
from torch.utils.data import DataLoader
import matplotlib.pyplot as plt
```

Στην συνέχεια ορίζεται η κλάση μας "NeuralNetwork" η οποία κληρονομεί την κλάση "nn.Module" του Pytorch.

Στον constructor της κλάσης μας, ορίζουμε την αρχιτεκτονική του Νευρωνικού μας. Ορίζονται τα τρία επίπεδα του δικτύου. Πιο συγκεκριμένα, το πρώτο layer, είναι ένα γραμμικό επίπεδο με είσοδο "input_size" και έξοδο "hidden_size". Στο επίπεδο αυτό, συνδέεται κάθε είσοδος με μια έξοδο με ένα αντίστοιχο βάρος, και προσθέτει το bias.

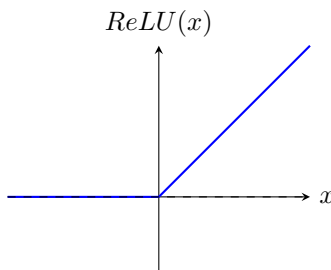
Το "hidden_size" αποτελεί το κρυφό επίπεδο του δικτύου! Ανάλογα με το μέγεθος που έχει αυτή η παράμετρος, αλλάζει και ο αριθμός των νευρώνων στο κρυφό επίπεδο!

Μετά το κρυφό επίπεδο, εφαρμόζεται η συνάρτηση ενεργοποίησης ReLU (Rectified Linear Unit). Στην συνέχεια, τα δεδομένα προωθούνται μέσα από το δεύτερο γραμμικό επίπεδο του δικτύου και επιστρέφεται

η έξοδος του.

Έπειτα, υλοποιείται η συνάρτηση "forward" η οποία είναι υπεύθυνη για τον τρόπο με τον οποίο προωθούνται τα δεδομένα μέσα στο νευρωνικό (forward pass). Δουλεύει με τον εξής τρόπο. Αλλάζει το σχήμα της εισόδου, ώστε να είναι δισδιάστατη, (η πρώτη διάσταση είναι το μέγεθος της παρτίδας (batch), και η δεύτερη είναι -1, το οποίο αντιστοιχεί στο "flatten" των εισόδων), προωθεί τα δεδομένα μέσα από το πρώτο γραμμικό επίπεδο, εφαρμόζει την συνάρτηση ReLU (βλ. παρακάτω) και τελικά προωθεί τα δεδομένα μέσα από το δεύτερο γραμμικό επίπεδο.

Figure 1.1: ReLU activation function



```
class NeuralNetwork(nn.Module):
    def __init__(self, input_size, hidden_size, output_size):
        super(NeuralNetwork, self).__init__()
        self.layer1 = nn.Linear(input_size, hidden_size)
        self.relu = nn.ReLU()
        self.layer2 = nn.Linear(hidden_size, output_size)

    def forward(self, x):
        x = x.view(x.size(0), -1) #flatten the input
        x = self.layer1(x)
        x = self.relu(x)
        x = self.layer2(x)
        return x
```

Αφού ορίσαμε την κλάση μας, ορίζουμε την συνάρτηση `train_model` η οποία είναι υπεύθυνη για την εκπαίδευση (training) και τον έλεγχο (testing) του νευρωνικού μας δικτύου. Αρχικά ορίζουμε κάποια arrays τα οποία θα μας φανούν χρήσιμα στην συνέχεια για την έκθεση των αποτελεσμάτων μας. Έπειτα, εκπαιδεύουμε το μοντέλο μας για έναν συγκεκριμένο αριθμό εποχών, (ο οποίος θα δούμε στην συνέχεια πως επηρεάζει την ακρίβεια του νευρωνικού μας δικτύου) υπολογίζοντας το κόστος, και εφαρμόζουμε τον back propagation. Ταυτόχρονα εκτελούμε και κάποια βήματα βελτιστοποίησης. Συνεχίζουμε ελέγχοντας πόσες "σωστές" προβλέψεις έχει κάνει το νευρωνικό μας σε κάθε εποχή, προκειμένου να υπολογίσουμε την ακρίβεια του νευρωνικού μας δικτύου, την οποία επιστρέφουμε στο τέλος της συνάρτησης. Τέλος, κρατάμε ενδεικτικά 5 σωστά και 5 λανθασμένα παραδείγματα κατηγοριοποίησης προκειμένου να τα εκτυπώσουμε μετά το πέρας της εκπαίδευσης του δικτύου μας. Ακόμα, μετά το τέλος της εκπαίδευσης του δικτύου, εκτυπώνουμε τον συνολικό χρόνο εκπαίδευσης (training time).

```

def train_model(model, train_loader, criterion, optimizer, epochs):
    accuracy_history = []
    correctly_classified_images = []
    incorrectly_classified_images = []
    images_to_be_displayed = 5
    #Record the start time
    start_time = time.time()

    for epoch in range(epochs):
        model.train()
        for inputs, labels in train_loader:
            optimizer.zero_grad()
            outputs = model(inputs)
            loss = criterion(outputs, labels)
            loss.backward()
            optimizer.step()
        # Test the model and calculate accuracy
        model.eval()
        correct = 0
        total = 0

        with torch.no_grad():
            for inputs, labels in test_loader:
                outputs = model(inputs)
                _, predicted = torch.max(outputs, 1)
                total += labels.size(0)
                correct += (predicted == labels).sum().item()
            # Track correct and incorrect classifications only after
            #the last epoch
            if epoch == epochs - 1:
                for i in range(labels.size(0)):
                    image = inputs[i].numpy().transpose((1, 2, 0))
                    actual_class = cifar_test.classes[labels[i]]
                    predicted_class = cifar_test.classes[predicted[i]]

                    if predicted[i] == labels[i] and
                    len(correctly_classified_images) <
                    images_to_be_displayed:
                        correctly_classified_images.append
                        ((image, actual_class, predicted_class))
                    elif predicted[i] != labels[i] and
                    len(incorrectly_classified_images) <
                    images_to_be_displayed:
                        incorrectly_classified_images.append
                        ((image, actual_class, predicted_class))

        accuracy = correct / total
        accuracy_history.append(accuracy)

    print(f"Epoch-{epoch+1}/{epochs},
    -----Accuracy on the test set: {100*accuracy:.2f}%")

```

```

#Record the end time
end_time = time.time()

#Calculate and print the training time
training_time = end_time - start_time
print(f"Training-time: -{training_time:.2f}-seconds")

display_images(correctly_classified_images ,
               "Correctly-Classified-Examples")
display_images(incorrectly_classified_images ,
               "Incorrectly-Classified-Examples")
return accuracy_history

```

Το επόμενο κομμάτι του κώδικα, αναφέρεται στις παραμέτρους του μοντέλου μας, ενδεικτικά στο μέγεθος εισόδου, το οποίο είναι 3072 (συνολικός αριθμός των pixels της κάθε εικόνας) και το μέγεθος εξόδου το οποίο είναι 10, όσες δηλαδή και οι κλάσεις του dataset. Ο αριθμός των κρυφών νευρώνων όπως θα δούμε και παρακάτω είναι μια από τις παραμέτρους που επηρεάζει την ακρίβεια του Νευρωνικού μας δικτύου! Τέλος επιλέγεται το loss function Cross Entropy Loss.

```

# Hyperparameters
input_size = 3 * 32 * 32
output_size = 10

criterion = nn.CrossEntropyLoss()

```

Στην συνέχεια του κώδικα, ασχολούμαστε με τον χειρισμό και την "προετοιμασία" του dataset. Όπως αναφέρθηκε και παραπάνω, το επιλεγμένο dataset είναι το Cifar10. Αυτό κατεβαίνει τοπικά με την βοήθεια του pytorch στον υπολογιστή μας και χωρίζεται σε test και train batches. Επίσης ορίζουμε μια μεταβλήτη "transform" (στην οποία αναθέτουμε μια συνάρτηση), η οποία θα είναι υπεύθυνη για τον μετασχηματισμό των εικόνων του dataset πριν αυτές χρησιμοποιηθούν στο νευρωνικό δίκτυο. Μετατρέπει αρχικά την εικόνα σε tensor, και στην συνέχεια κανονικοποιεί τις τιμές των pixels του tensor με μέση τιμή 0.5 και διακύμανση 0.5 για κάθε κανάλι χρώματος (R,G,B). Αυτό θα έχει ως αποτέλεσμα τα pixels της εικόνας, να έχουν μετά τον μετασχηματισμό, τιμές που βρίσκονται περίπου στο εύρος [-1, 1].

Η κανονικοποίηση (normalization) στο πλαίσιο της μηχανικής μάθησης αναφέρεται στη διαδικασία της προσαρμογής των τιμών ενός χαρακτηριστικού (ή πολλών χαρακτηριστικών) έτσι ώστε να έχουν μέση τιμή 0 και διακύμανση 1.

Τέλος δημιουργούμε έναν DataLoader, προκειμένου να διαχειριζόμαστε πιο αποτελεσματικά τα δεδομένα μας κατά την διάρκεια της εκπαίδευσης και του ελέγχου του νευρωνικού μας δικτύου.

```

#Defining the transformation that is gonna be applied to the images
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
])

#Downloading the dataset
cifar_train = datasets.CIFAR10(root='./data', train = True,
download = True, transform = transform)
cifar_test = datasets.CIFAR10(root = './data', train = False,
download = True, transform = transform)

#Configuring the dataloader —> beneficial to use it, it helps
to load and iterate over the dataset efficiently
train_loader = torch.utils.data.DataLoader(cifar_train, batch_size=64,
shuffle = True, num_workers = 2)
test_loader = torch.utils.data.DataLoader(cifar_test, batch_size=64,
shuffle = False, num_workers = 2)

```

Η επόμενη συνάρτηση, μας επιτρέπει να εμφανίσουμε κάποιες επιλεγμένες εικόνες αφού εκπαιδύσουμε το νευρωνικό μας δίκτυο, προκειμένου να εξετάσουμε κάποια παραδείγματα ορθής και εσφαλμένης κατηγοριοποίησης.

```

# Function to display images
def display_images(images, title):
    num_images = len(images)
    fig, axes = plt.subplots(1, num_images, figsize=(12, 4))

    for i, (image, actual_class, predicted_class) in enumerate(images):
        axes[i].imshow(image)
        axes[i].axis('off')
        axes[i].set_title(f'Actual: -{actual_class}\nPredicted:
        -----{predicted_class}')

    fig.suptitle(title)
    plt.show()

```

Στο τελευταίο κομμάτι του κώδικα μας, πειραματιζόμαστε με διάφορες τιμές τόσο για τον αριθμό των κρυφών νευρώνων, όσο και για τα learning rates, και δημιουργούμε γραφήματα που δείχνουν την σχέση των εποχών με το accuracy του δικτύου μας για την εκάστοτε τιμή.

```

#Experiment with different hidden layer sizes
hidden_sizes = [50, 100, 200, 300]

for hidden_size in hidden_sizes:
    model = NeuralNetwork(input_size, hidden_size, output_size)
    optimizer = optim.SGD(model.parameters(), lr=0.01)
    accuracy_history = train_model(model, train_loader,
    criterion, optimizer, epochs=30)
    plot_accuracy(accuracy_history, f'Accuracy~with~Hidden~Size~==
    ~~~{hidden_size}~')

#Experiment with different learning rates
learning_rates = [0.001, 0.005, 0.01, 0.015, 0.02, 0.05, 0.1]

for learning_rate in learning_rates:
    model = NeuralNetwork(input_size, 300,output_size)
    optimizer = optim.SGD(model.parameters(), lr = learning_rate)
    accuracy_history = train_model(model, train_loader, criterion,
    optimizer, epochs = 30)
    plot_accuracy(accuracy_history, f'Accuracy~with~learning~rate~==
    ~~~~{learning_rate}~')

```


Chapter 2

Έκθεση των αποτελεσμάτων

Σε αυτό το κεφάλαιο, θα παρουσιαστούν κάποια αποτελέσματα που προέκυψαν από την δημιουργία του Νευρωνικού μας δικτύου. Αναλυτικότερα θα γίνει αναφορά:

- σε μερικά παραδείγματα ορθής και εσφαλμένης κατηγοριοποίησης
- στα ποσοστά επιτυχίας του νευρωνικού μας δικτύου, σε διάφορα στάδια της εκπαίδευσης
- στον χρόνο που χρειάζεται το νευρωνικό μας δίκτυο για να εκπαιδευτεί
- στα διαφορετικά ποσοστά ακριβείας που επιτυγχάνουμε με μεταβολή κάποιων παραμέτρων εκπαίδευσης

Τέλος, θα γίνει εν συντομία και μια σύγκριση μεταξύ της απόδοσης του νευρωνικού δικτύου, σε σχέση με τους κατηγοριοποιητές πλησιέστερου γείτονα (KNN) και πλησιέστερου κέντρου (Nearest Centroid) της ενδιάμεσης εργασίας.

2.1 Ορθή - Εσφαλμένη κατηγοριοποίηση

Οι παρακάτω κατηγοριοποιήσεις, αντιστοιχούν σε υλοποίηση του δικτύου μας με $learning\ rate = 0.015$, 300 κρυφούς νευρώνες, και 40 εποχές. Παρακάτω, παρουσιάζονται με την σειρά:

- 5 παραδείγματα ορθής κατηγοριοποίησης του νευρωνικού μας δικτύου.
- 5 παραδείγματα εσφαλμένης κατηγοριοποίησης του νευρωνικού μας δικτύου.
- το ποσοστό επιτυχίας του δικτύου μας με το πέρας των εποχών και το σχετικό γράφημα.

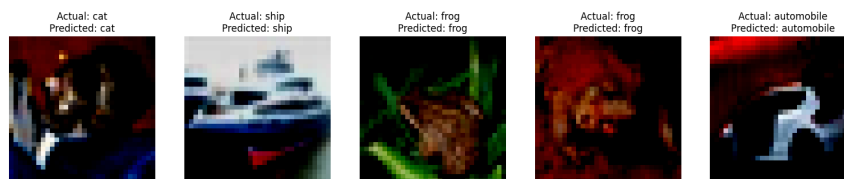


Figure 2.1: Correctly Classified Images



Figure 2.2: Incorrectly Classified Images

```

Epoch 1/40, Accuracy on the test set: 42.39%
Epoch 2/40, Accuracy on the test set: 45.53%
Epoch 3/40, Accuracy on the test set: 46.63%
Epoch 4/40, Accuracy on the test set: 48.82%
Epoch 5/40, Accuracy on the test set: 48.95%
Epoch 6/40, Accuracy on the test set: 50.79%
Epoch 7/40, Accuracy on the test set: 51.22%
Epoch 8/40, Accuracy on the test set: 50.31%
Epoch 9/40, Accuracy on the test set: 51.91%
Epoch 10/40, Accuracy on the test set: 52.41%
Epoch 11/40, Accuracy on the test set: 50.89%
Epoch 12/40, Accuracy on the test set: 50.44%
Epoch 13/40, Accuracy on the test set: 52.05%
Epoch 14/40, Accuracy on the test set: 51.14%
Epoch 15/40, Accuracy on the test set: 51.03%
Epoch 16/40, Accuracy on the test set: 51.24%
Epoch 17/40, Accuracy on the test set: 52.86%
Epoch 18/40, Accuracy on the test set: 52.48%
Epoch 19/40, Accuracy on the test set: 50.55%
Epoch 20/40, Accuracy on the test set: 52.48%
Epoch 21/40, Accuracy on the test set: 52.44%
Epoch 22/40, Accuracy on the test set: 52.75%
Epoch 23/40, Accuracy on the test set: 52.08%
Epoch 24/40, Accuracy on the test set: 51.97%
Epoch 25/40, Accuracy on the test set: 52.71%
Epoch 26/40, Accuracy on the test set: 52.16%
Epoch 27/40, Accuracy on the test set: 53.45%
Epoch 28/40, Accuracy on the test set: 51.75%
Epoch 29/40, Accuracy on the test set: 52.29%
Epoch 30/40, Accuracy on the test set: 51.34%
Epoch 31/40, Accuracy on the test set: 51.11%
Epoch 32/40, Accuracy on the test set: 51.48%
Epoch 33/40, Accuracy on the test set: 51.54%
Epoch 34/40, Accuracy on the test set: 51.15%
Epoch 35/40, Accuracy on the test set: 52.41%
Epoch 36/40, Accuracy on the test set: 51.36%
Epoch 37/40, Accuracy on the test set: 51.14%
Epoch 38/40, Accuracy on the test set: 51.98%
Epoch 39/40, Accuracy on the test set: 50.90%
Epoch 40/40, Accuracy on the test set: 52.62%
Training time: 221.77 seconds

```

Figure 2.3: Accuracy after each epoch

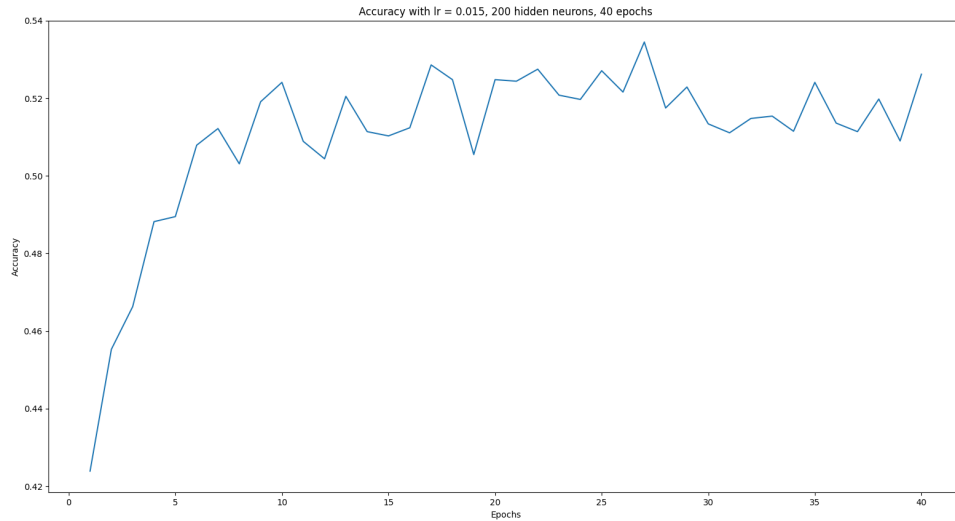


Figure 2.4: Accuracy over epochs

Όπως είναι φυσικό, το νευρωνικό μας δίκτυο δεν μπορεί να κατηγοριοποιήσει όλες τις εικόνες με ορθό τρόπο. Βλέπουμε πως μετά την διαδικασία της εκπαίδευσης, το μοντέλο μας φτάνει να έχει ακρίβεια της τάξης του 52.62%. Είναι αναμενόμενο λοιπόν να βρεθούν εικόνες οι οποίες αντιστοιχήθηκαν σε λάθος κλάση, και παρουσιάζονται μερικές από αυτές! Αξίζει επίσης να σημειωθεί, πως στο γράφημα της ακρίβειας από ένα σημείο και μετά παρατηρούνται αιχμές, δηλαδή η ακρίβεια αυξομειώνεται μετά από κάθε εποχή. Αυτό μπορεί να οφείλεται στην επιλογή του learning rate, στον αριθμό των κρυφών νευρώνων, αλλά και στο μέγεθος του batch που εισάγεται στο νευρωνικό! Ενδεικτικά, παρατίθεται η αντίστοιχη γραφική παράσταση για learning rate = 0.01 και 200 κρυφούς νευρώνες, η οποία όπως παρατηρούμε είναι πιο "ομαλή".

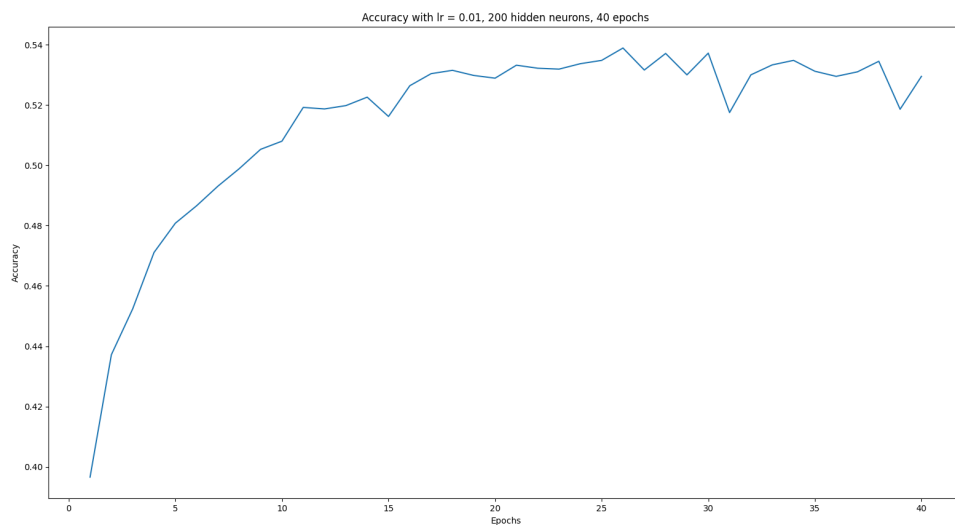


Figure 2.5: Accuracy Over epochs

2.2 Εκπαίδευση του Νευρωνικού με διαφορετικό αριθμό νευρώνων στο κρυφό επίπεδο

Στο παρακάτω πίνακάκι, παραθέτονται ο χρόνος εκπαίδευσης του δικτύου, καθώς και η ακρίβεια που πετυχαίνουμε για διαφορετικές τιμές της μεταβλητής `hidden_size` που δηλώνει τον αριθμό των νευρώνων στο κρυφό επίπεδο.

Οι παρακάτω μετρήσεις έγιναν με 30 εποχές και $learning\ rate = 0.01$

Neurons	Training Time	Accuracy
50	140.96	49.91%
100	138.6s	51.77%
200	142.95s	52.36%
300	145.06s	53.99%
400	146.79s	53.83%
500	151.2s	53.52%

Table 2.1

Παρατηρούμε πως ο χρόνος εκπαίδευσης του νευρωνικού μας δικτύου μεγαλώνει με την αύξηση των νευρώνων στο κρυφό επίπεδο. Αυτό μπορεί να οφείλεται σε διαφορετικούς λόγους. Αρχικά, με περισσότερους νευρώνες, αυξάνεται η υπολογιστική πολυπλοκότητα του δικτύου, καθώς υπάρχουν περισσότερες παράμετροι που πρέπει να ενημερώνονται κατά τη διάρκεια της εκπαίδευσης και απαιτούνται περισσότερες λειτουργίες για τον υπολογισμό των κλίσεων και των βαρών.

Ακόμα, ένα μεγαλύτερο κρυφό στρώμα απαιτεί περισσότερη μνήμη για την αποθήκευση ενδιάμεσων τιμών κατά την προώθηση (forward) και την πίσω προώθηση (back propagation). Αυτή η αυξημένη απαίτηση μνήμης μπορεί να οδηγήσει σε μεγαλύτερους χρόνους εκπαίδευσης, ειδικά αν το υλικό σας έχει περιορισμούς στη διαθέσιμη μνήμη.

Τέλος, μεγαλύτερα κρυφά στρώματα μπορεί να οδηγήσουν σε έναν πιο πολύπλοκο χώρο απώλειας, που πιθανόν να οδηγήσει σε πιο αργή σύγκλιση κατά τη βελτιστοποίηση. Ο αλγόριθμος βελτιστοποίησης (optimizer) ενδέχεται να απαιτεί περισσότερα βήματα για να βρει το ελάχιστο της συνάρτησης απώλειας (loss function).

Όσον αφορά την ακρίβεια, παρατηρούμε πως αρχικά με την αύξηση των νευρώνων, βελτιώνεται η ακρίβεια του νευρωνικού, ενώ στην συνέχεια (μετά τους 300 κρυφούς νευρώνες) η ακρίβεια "σταθεροποιείται". Κάτι τέτοιο συμβαίνει πολύ πιθανόν λόγω της υπερεκπαίδευσης (overfitting). Αυτό σημαίνει πως καθώς αυξάνεται ο αριθμός των κρυφών νευρώνων, το μοντέλο γίνεται πιο πολύπλοκο και μπορεί να αρχίσει να μαθαίνει το θόρυβο των δεδομένων εκπαίδευσης, αντί για γενικά χαρακτηριστικά.

Ακολουθούν τα διαγράμματα της ακρίβειας για το κάθε αριθμό των κρυφών νευρώνων με την πάροδο των εποχών.

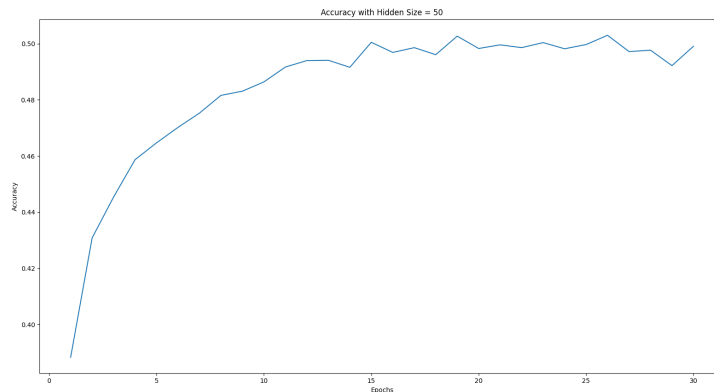


Figure 2.6: 50 Neurons

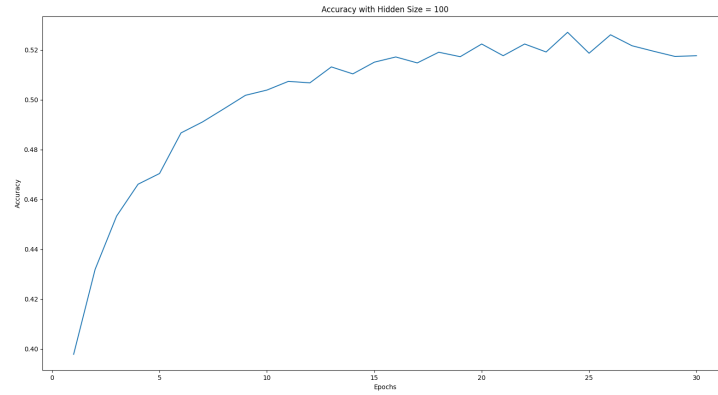


Figure 2.7: 100 Neurons

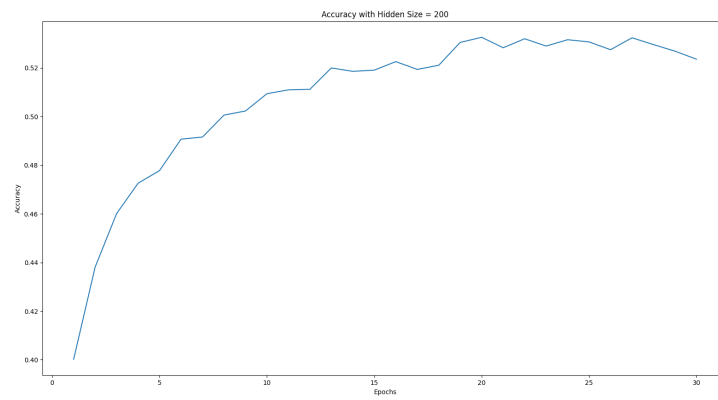


Figure 2.8: 200 Neurons

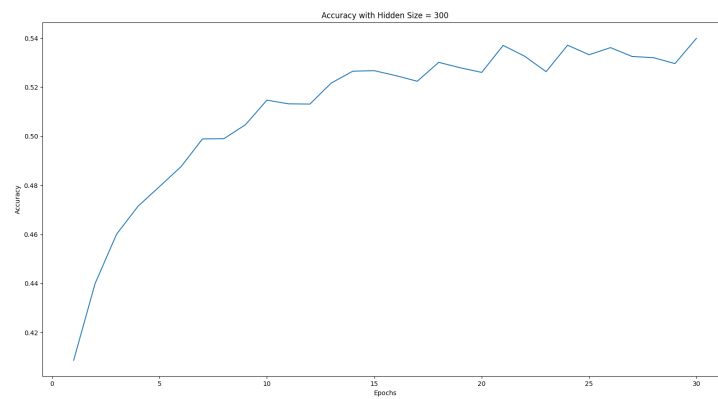


Figure 2.9: 300 Neurons

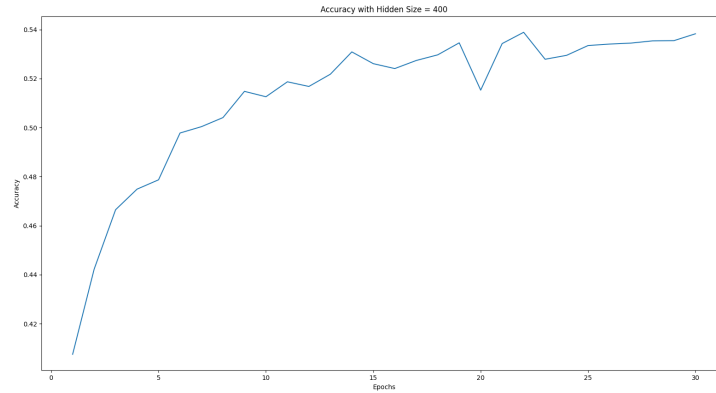


Figure 2.10: 400 Neurons

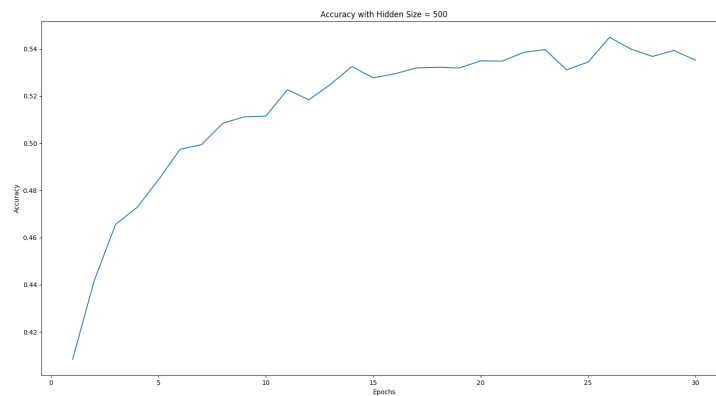


Figure 2.11: 500 Neurons

2.3 Επίδραση της τιμής του learning rate στην ακρίβεια και στον χρόνο εκπαίδευσης του νευρωνικού

Στο παρακάτω πίνακάκι, παραθέτονται ο χρόνος εκπαίδευσης του δικτύου, καθώς και η ακρίβεια που πετυχαίνουμε για διαφορετικές τιμές της μεταβλητής $lr = \text{learning rate}$.

Οι παρακάτω μετρήσεις έγιναν με 30 εποχές και 300 νευρώνες στο κρυφό επίπεδο

Learning rate	Training time	Accuracy
0.001	144.96s	46.5%
0.005	146.94s	53.32%
0.01	148.3s	53.22%
0.015	148.26s	50.27%
0.02	147.33s	49.76%
0.05	145.46s	41.98%
0.1	147.46s	47.97%

Table 2.2

Παρατηρούμε πως ο χρόνος εκπαίδευσης αυτή την φορά, παραμένει "σχετικά" σταθερός με την αλλαγή του learning rate και δεν μπορεί να βγει κάποιο συγκεκριμένο συμπέρασμα.

Δεν μπορούμε να πούμε όμως το ίδιο και για την ακρίβεια του δικτύου. Πιο συγκεκριμένα, παρατηρούμε πως όσο αυξάνουμε το learning rate, μεγαλώνει η ακρίβεια του νευρωνικού, μέχρι ένα συγκεκριμένο "κατώφλι" (learning rate = 0.01). Από αυτό το σημείο και μετά το accuracy ξεκινάει και πάλι να μειώνεται! Αυτό μπορεί να συμβεί για διάφορους λόγους:

- Όταν αυξάνεται το learning rate, τα βήματα βελτιστοποίησης είναι μεγαλύτερα, και το μοντέλο μπορεί να συγκλίνει πιο γρήγορα προς ένα τοπικό ελάχιστο. Ωστόσο, εάν το learning rate γίνει πολύ μεγάλο, το νευρωνικό μας μπορεί να υπερπηδήσει το ελάχιστο και να αρχίσουμε να απομακρυνόμαστε από τη βέλτιστη λύση.
- Καθώς αυξάνεται το learning rate, το μοντέλο μπορεί να προσαρμόζεται περισσότερο στα δεδομένα εκπαίδευσης, προκαλώντας υπερεκπαίδευση. Αυτό μπορεί να οδηγήσει σε μείωση της γενικής ικανότητας του μοντέλου να γενικεύει σε νέα δεδομένα. (overfitting)
- Υψηλά learning rates μπορεί να οδηγήσουν σε αστάθεια κατά τη σύγκλιση. Αν η τιμή του learning rate είναι υπερβολικά μεγάλη, τα βάρη μπορεί να καταλήξουν να αλλάζουν υπερβολικά, και το μοντέλο μπορεί να μη συγκλίνει σε μια σταθερή λύση.

Ακολουθούν τα διαγράμματα της ακρίβειας για κάθε διαφορετική τιμή του learning rate. (στα οποία μπορούμε να αντιληφθούμε την "αστάθεια" που αναφέρθηκε παραπάνω για μεγάλα learning rates από τις μεγάλες αυξομειώσεις που παρατηρούνται στην ακρίβεια μετά από κάθε εποχή).

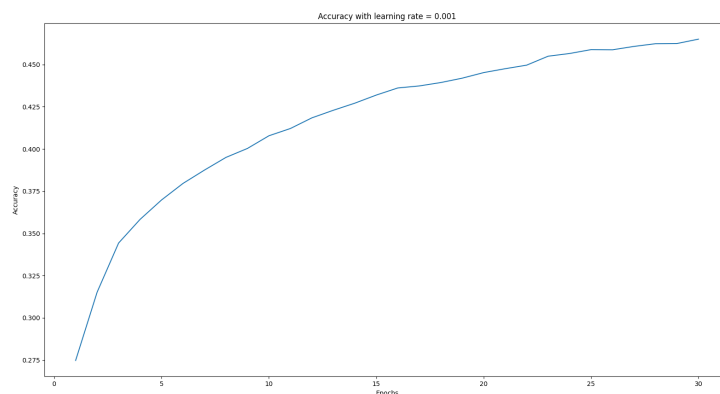


Figure 2.12: Learning rate = 0.001

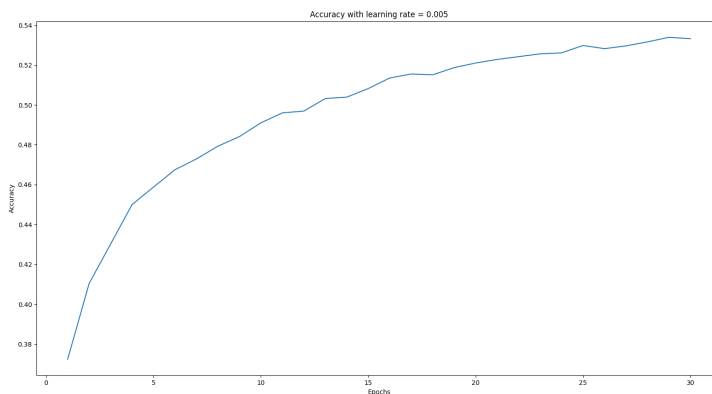


Figure 2.13: Learning rate = 0.005

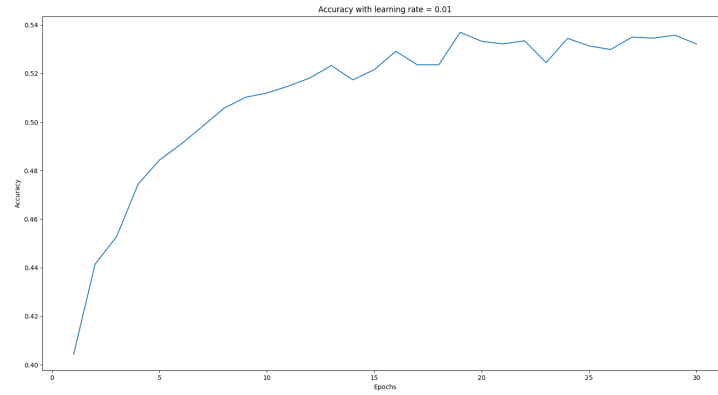


Figure 2.14: Learning rate = 0.01

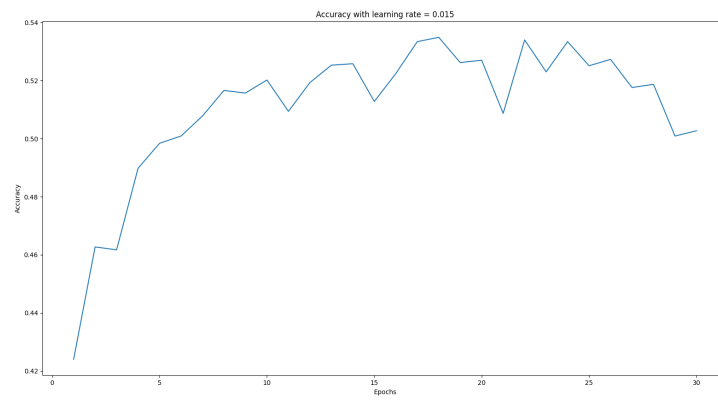


Figure 2.15: Learning rate = 0.015

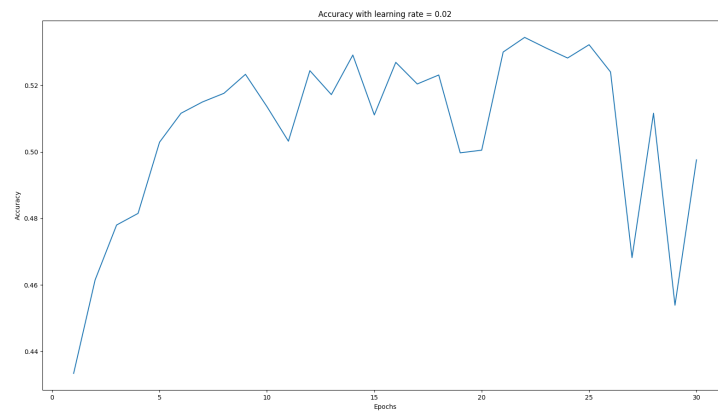


Figure 2.16: Learning rate = 0.02

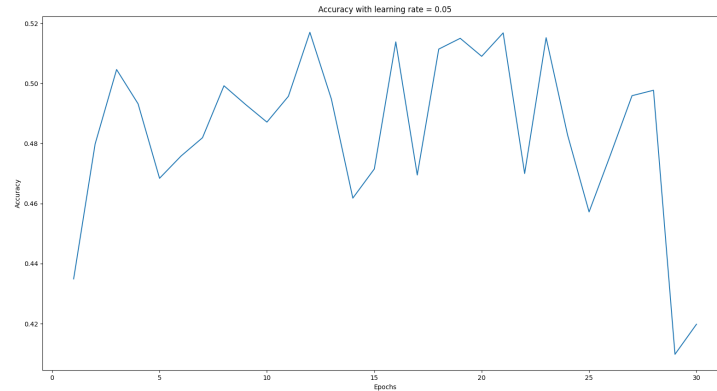


Figure 2.17: Learning rate = 0.05

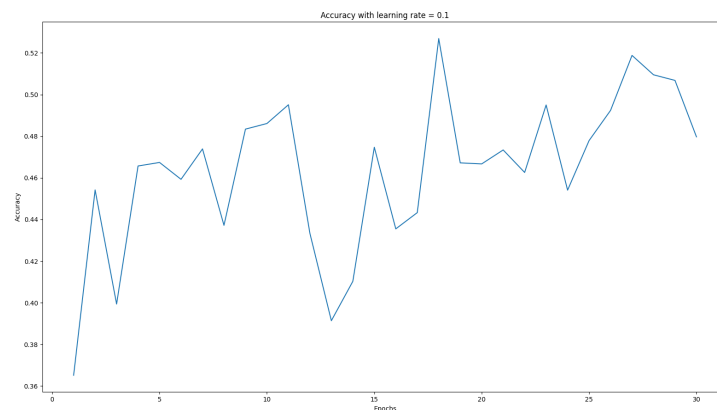


Figure 2.18: Learning rate = 0.1

2.4 Σύγκριση του νευρωνικού μας δικτύου με τους κατηγοριοποιητές KNN και KNC

2.4.1 K-Nearest Neighbor Classifier

Ο κατηγοριοποιητής K πλησιέστερου γείτονα, είναι ένας διάσημος αλγόριθμος μηχανικής μάθησης, ο οποίος μπορεί να διαχειριστεί προβλήματα regression και classification. Κατά την διάρκεια της φάσης εκπαίδευσης, ο KNN αποθηκεύει το dataset σαν αναφορά. Όταν έρχεται η ώρα να κάνει προβλέψεις, με βάση μια συγκεκριμένη ιδιότητα που θεωρεί ως απόσταση (συνήθως την ευκλείδεια απόσταση), υπολογίζει την απόσταση ανάμεσα στα δεδομένα που του δίνουμε ως input, και όλα τα παραδείγματα εκπαίδευσης. Έπειτα, ο αλγόριθμος ταυτοποιεί τους K (στην περίπτωση μας 1, 3) πλησιέστερους γείτονες στο input. Αν το πρόβλημα μας είναι κατηγοριοποίησης ο αλγόριθμος αναθέτει την πιο συχνή κλάση που συναντάει στους γείτονες ως την πρόβλεψη του για το input data.

2.4.2 Nearest Centroid Classifier

Ο κατηγοριοποιητής πλησιέστερου κέντρου, είναι και αυτός ένας αλγόριθμος μηχανικής μάθησης, τον οποίο μπορούμε να εφαρμόσουμε σε προβλήματα κατηγοριοποίησης. Η κάθε κλάση του dataset, αναπαριστάται από το κέντρο της (δηλαδή τον "μέσο όρο" μιας τιμής που επιλέξαμε). Έτσι, όταν περάσουμε

στον κατηγοριοποιητή κάποιο input, αυτός θα το αναθέσει σύμφωνα με την κλάση στην οποία ανήκει το πλησιέστερο κέντρο από το input data.

Αφού υλοποίησα λοιπόν τους παραπάνω κατηγοριοποιητές στο περιβάλλον της *python*, συνέκρινα την ακρίβεια τους, και όπως είναι φανερό από τον πίνακα παρακάτω, ο κατηγοριοποιητής πλησιέστερου γείτονα για $k = 1$, είναι ο πιο αποτελεσματικός και ακριβής, ενώ ακολουθούν με την σειρά ο κατηγοριοποιητής πλησιέστερου γείτονα για $k = 3$ και ο κατηγοριοποιητής πλησιέστερου κέντρου

Αλγόριθμος	Accuracy
KNN ($k = 1$)	35.39%
KNN ($k=3$)	33.03%
KNC	27.74%

Όπως είναι φανερό, η ακρίβεια του νευρωνικού μας δικτύου είναι αρκετά μεγαλύτερη από αυτήν των κατηγοριοποιητών της ενδιάμεσης εργασίας. Πιο συγκεκριμένα, παρατηρείται μια διαφορά στην ακρίβεια της τάξης 20%!

Αυτό είναι φυσιολογικό και αναμενόμενο. Το νευρωνικό μας δίκτυο περιμέναμε να ξεπεράσει σε ακρίβεια τους υπόλοιπους κατηγοριοποιητές για τους παρακάτω λόγους.

- Τα νευρωνικά δίκτυα εισάγουν από μόνα τους μη γραμμικότητα μέσω συναρτήσεων ενεργοποίησης στα επίπεδά τους. Αυτό τους επιτρέπει να μοντελοποιούν πολύπλοκες σχέσεις μεταξύ χαρακτηριστικών και ετικετών. Αντίθετα, το KNN και ο Κοντινότερος Κέντρος είναι γραμμικοί ταξινομητές και ενδέχεται να δυσκολεύονται να αντιληφθούν μη γραμμικά πρότυπα στα δεδομένα.
- Τα νευρωνικά δίκτυα συχνά ωφελούνται από μεγάλα σύνολα δεδομένων. Έχουν τη δυνατότητα να μαθαίνουν από ένα μεγάλο αριθμό παραδειγμάτων και να γενικεύουν καλά σε μη ορατά δεδομένα. Αντίθετα, ο KNN, που βασίζεται σε ολόκληρο το σύνολο δεδομένων κατά την πρόβλεψη, ενδέχεται να μην κλιμακώνεται καλά με ένα μεγάλο αριθμό παραδειγμάτων.
- Το KNN έχει υπερπαραμέτρους, όπως ο αριθμός των γειτόνων (k), που πρέπει να ρυθμιστούν. Η επιλογή του k μπορεί να επηρεάσει την απόδοση, και η εύρεση της βέλτιστης τιμής απαιτεί πειραματισμό. Τα νευρωνικά δίκτυα έχουν επίσης υπερπαραμέτρους, αλλά μπορεί να είναι πιο συγκρίσιμες ή πιο εύκολες να ρυθμιστούν σε ορισμένα σενάρια.
- Τα νευρωνικά δίκτυα είναι πολύ εκφραστικά μοντέλα με έναν μεγάλο αριθμό παραμέτρων, επιτρέποντάς τους να καταγράφουν περίπλοκα πρότυπα στα δεδομένα. Αυτή η δυνατότητα μπορεί να είναι επωφελής για περίπλοκα σύνολα δεδομένων. Το KNN και ο Nearest centroid, ως πιο απλά μοντέλα, ενδέχεται να δυσκολεύονται να αντιμετωπίσουν τέτοια περίπλοκα πρόβληματα.